

Winning image restoration with only 2,400 training pairs

Fine-tuning a pretrained transformer model on your small dataset — rather than training from scratch — is the single highest-impact change you can make, likely worth +1–3 dB PSNR alone. Combined with CutBlur augmentation, online degradation diversity, a Charbonnier + SSIM loss combination, and test-time self-ensemble, this strategy can realistically close much of the gap between your current 0.6547 score and the leader's 0.78. The research is clear: every recent NTIRE/AIM restoration competition winner used pretrained models, [arXiv](#) ↗ aggressive augmentation, and ensembling [The Moonlight](#) ↗ — and with only 2,400 pairs, these techniques yield disproportionately large gains compared to architectural tweaks.

Pretrained fine-tuning is your biggest lever

Training a NAFNet-style U-Net from scratch on 2,400 pairs severely underutilizes available knowledge. The restoration community has pretrained checkpoints that encode millions of image priors, and fine-tuning them on small datasets consistently outperforms from-scratch training by **1–3 dB PSNR**.

SwinIR is the recommended starting point for this specific task. It offers both grayscale denoising checkpoints (`004_grayDN_DF2K_s128w8_SwinIR-M_noise25.pth`) that accept 1-channel input directly, and classical $2\times$ SR checkpoints (`001_classicalSR_DF2K_s64w8_SwinIR-M_x2.pth`) trained on DIV2K+Flickr2K. The grayscale denoising checkpoint is particularly valuable because it requires zero channel adaptation — load it, modify the upsampler for $2\times$ upscaling, and fine-tune. At **11.8M parameters**, SwinIR-M is small enough to train comfortably within a 12-hour Kaggle budget. [Emergent Mind](#) ↗ [TheCVF](#) ↗

HAT (Hybrid Attention Transformer) is the alternative if you want maximum ceiling performance. It provides `HAT_SRx2_ImageNet-pretrain.pth` specifically for $2\times$ SR, and a Kaggle dataset hosts these weights directly. However, HAT checkpoints are RGB-only, requiring adaptation: either replicate your grayscale channel $3\times$ as input (`x.repeat(1, 3, 1, 1)`), or average the first convolution's 3-channel weights into a single channel (`new_weight = old_weight.mean(dim=1, keepdim=True)`). [GitHub](#) ↗ Both approaches preserve pretrained representations effectively.

Restormer deserves mention for its remarkable fine-tuning efficiency. At NTIRE 2025, the 6th-place denoising team achieved competitive results using **bias-only tuning** — freezing all weights and training only bias parameters [The Moonlight](#) ↗ ($\sim 0.1\%$ of total parameters). [The Moonlight](#) ↗ This extreme parameter efficiency makes Restormer a strong fallback if overfitting proves severe on 2,400 images.

The fine-tuning recipe across all models is consistent: use **AdamW** with initial learning rate 2×10^{-5} (roughly 1/10th of from-scratch LR), **2,000 iterations of linear warmup**, and **cosine annealing** down to 1×10^{-7} over 100K–200K total iterations. Apply weight decay of 1×10^{-4} and stochastic depth of 0.1–0.2 to combat overfitting. Save checkpoints frequently for later ensemble averaging.

CutBlur and geometric augmentation deliver outsized returns on small datasets

Not all augmentations transfer from classification to restoration — in fact, many popular ones actively harm pixel-level tasks. **CutMix, Cutout, ManifoldMixup, and ShakeDrop all degrade SR performance** because they destroy the spatial correspondence between input and target that restoration requires. [AI-SCHOLAR](#) ↗ The research

is unambiguous: only augmentations preserving structural integrity help. [arXiv](#)

The augmentation priority stack, ranked by impact:

1. **Geometric self-augmentation (8× multiplier, zero cost).** Random horizontal flip, vertical flip, and 90°/180°/270° rotation applied identically to both LR and HR. Every serious restoration paper uses these. They are mandatory.
2. **CutBlur (~+0.2–0.4 dB, highest ROI for small data).** This technique cuts a rectangular region from the LR image (upscaled to HR size) and pastes it into the HR image, and vice versa. [TheCVF](#) The model learns not just *how* to restore but *where* and *how much* to apply restoration, preventing over-sharpening. [TheCVF](#) [arXiv](#) The critical finding: **RCAN trained on 50% of DIV2K with CutBlur matches full-dataset performance without CutBlur.** [TheCVF](#) With 2,400 images, CutBlur's relative advantage is even larger. Implementation is available at github.com/clovaai/cutblur.
3. **MixUp for restoration pairs (+0.1–0.2 dB).** Blend two LR-HR pairs pixel-wise: $LR_{mix} = \lambda \cdot LR_1 + (1-\lambda) \cdot LR_2$ and identically for HR. Unlike CutMix, MixUp preserves structural information. The CutBlur paper confirms "mix-type" augmentations consistently outperform "cut-type" for restoration. [AI-SCHOLAR](#)
4. **Progressive patch sizing (+0.1–0.4 dB).** Train with 32×32 LR patches for the first 60% of iterations, then increase to 64×64. Recent work (EPTGC, 2025) shows this yields up to **+0.44 dB PSNR** on transformer architectures like SwinIR and HAT. No single patch granularity is optimal; the progressive mixture outperforms any fixed size. [Nature](#)
5. **Mixture of Augmentations (MoA).** Stochastically combine all the above each mini-batch. The CutBlur paper's MoA strategy — randomly selecting which augmentations to apply per batch — consistently achieves the best results across architectures. [TheCVF](#)

For grayscale-specific intensity augmentations, small brightness jitter (± 10 intensity levels) and contrast scaling ($0.9\text{--}1.1\times$) are safe **only if applied identically to both LR and HR**. Breaking this correspondence teaches the model incorrect mappings.

Online degradation diversity turns 2,400 pairs into effectively unlimited data

Rather than treating your 2,400 pairs as fixed, generate new degraded LR versions of each HR image on-the-fly every epoch. This single change transforms a data-starved pipeline into one with practically infinite variety.

The implementation is straightforward. For each HR image in a batch, apply a randomly sampled degradation chain: Gaussian blur (σ uniformly sampled from 0.5–3.0), additive Gaussian noise (σ from 5–25), downsampling via a randomly chosen method (bicubic, bilinear, or area), and optionally light JPEG compression (quality 70–95). Each time the image is sampled, different parameters produce a different LR, so the model never sees the exact same pair twice.

Real-ESRGAN's second-order pipeline takes this further by applying the degradation chain twice in sequence — first degradation produces an intermediate, second degradation produces the final LR. This generates substantially more complex and realistic artifacts. [arXiv](#) The specific sequence includes isotropic/anisotropic Gaussian blur kernels (sizes 7–21), random resize using area/bilinear/bicubic interpolation, Gaussian or Poisson noise injection, JPEG compression, then repeating with fresh random parameters. A sinc filter step simulates ringing artifacts. [arxiv](#) [TheCVF](#)

BSRGAN's shuffle strategy offers an alternative: instead of a fixed degradation order, it randomly permutes the sequence of {blur, downsample, noise, JPEG} operations. [Git Memory](#) This creates a far larger degradation space than fixed-order pipelines and improves generalization significantly. [GitHub](#)

A critical caveat applies to this competition. If your test degradation is well-defined and narrow (a specific blur + noise + downscaling pipeline), training on broader/harder degradations can actually **reduce** PSNR on that specific degradation. BSRGAN's own authors note this tradeoff. [Git Memory](#) [GitHub](#) The optimal

strategy: **analyze a few training pairs to estimate the actual degradation**, then center your synthetic pipeline around those parameters with moderate variance. If the degradation is unknown or variable, go broader.

External data amplifies this further. Convert DIV2K (800 images) and Flickr2K (2,650 images) to grayscale, [Medium](#) ↗ apply your estimated degradation pipeline, and use these for pretraining before fine-tuning on your 2,400 domain-specific pairs. [Internet Archive](#) ↗ NTIRE 2024 winners routinely pretrained on 85,000+ external images before domain fine-tuning. [arXiv](#) ↗

The right loss function combination can add +0.3–0.5 dB

Loss function selection has an outsized effect on PSNR/SSIM performance, and the research is remarkably clear about what works.

Charbonnier loss should be your primary pixel loss. Defined as $L = \sqrt{(y - \hat{y})^2 + \epsilon^2}$ with $\epsilon = 1 \times 10^{-3}$, it is a smooth approximation of L1 that eliminates gradient discontinuity at zero. SwinIR uses Charbonnier for denoising and JPEG artifact removal. [ResearchGate](#) ↗ [TheCVF](#) ↗ The foundational Zhao et al. (2017) study from NVIDIA demonstrated that **L1 consistently outperforms L2/MSE for PSNR** — counterintuitively, even though PSNR is mathematically derived from MSE. L2 produces splotchy artifacts in flat regions and gets stuck in worse local minima. [Medium](#) ↗

Combine Charbonnier with an SSIM component to directly optimize the competition metric. The landmark finding: MS-SSIM + L1 with weighting $\alpha=0.84$ achieves the **best results on all image quality metrics** tested, outperforming either loss alone. For grayscale images, SSIM loss is safer than for RGB because there are no color-shift issues (SSIM's main failure mode). Start training with Charbonnier only for ~1,000 iterations for stability, then add the SSIM component with weight 0.1–0.2.

Focal Frequency Loss (FFL) addresses spectral bias for high-frequency recovery. Neural networks preferentially learn low frequencies first (the "F-Principle"), and pixel losses cannot help the network locate hard-to-synthesize frequencies. FFL transforms images to the frequency domain via 2D DFT and applies adaptive weighting that focuses on under-reconstructed frequencies. [GitHub](#) ↗ Add FFL with weight 0.1–0.5 relative to the primary loss as a third component.

The recommended combined loss:



$$L_{\text{total}} = 1.0 \times \text{Charbonnier} + 0.15 \times (1 - \text{SSIM}) + 0.25 \times \text{FFL}$$

What to strictly avoid: Perceptual loss (VGG features) and LPIPS **reduce PSNR by 1–3 dB**. [arXiv](#) ↗ SRGAN's perceptual loss dropped PSNR from 30.62 to 27.64 on Set5 — a catastrophic loss for a PSNR/SSIM competition. The perception-distortion tradeoff (Blau & Michaeli, ICML 2018) is a fundamental theoretical result: you cannot simultaneously optimize distortion metrics and perceptual quality. [arXiv](#) ↗ For this competition, stay firmly on the distortion side. Also avoid L2/MSE as a primary loss and MS-SSIM alone from the start of training (NaN instability is widely reported by practitioners). [Faceswap Forum](#) ↗

Competition-winning ensemble and inference strategies

Every NTIRE/AIM winner employs some form of ensembling. [ResearchGate](#) ↗ [The Moonlight](#) ↗ The techniques range from zero-cost to moderate-cost, and they stack.

Self-ensemble (geometric TTA) is mandatory for PSNR competitions. Process each test image through all 8 geometric orientations (4 rotations $\times$ 2 flips), inverse-transform the outputs, and average. [TheCVF](#) ↗ This yields a consistent **+0.1–0.2 dB PSNR** improvement. EDSR+ won NTIRE 2017 using this technique. [Medium](#) ↗ The cost is 8 $\times$ inference time — acceptable if the scoring function doesn't heavily penalize speed, but since this competition weights inference time, you may want to limit TTA to 4 orientations (flips only, 4 $\times$ cost) if the time penalty is steep.

Checkpoint weight averaging (feature ensemble) is free at inference. Average the weights of 3–5 checkpoints saved during training (e.g., from the last 25% of training iterations). SwinFIR demonstrated **+0.08 dB** improvement with zero additional inference cost. [arXiv](#) ↗ This is strictly better than selecting a single best checkpoint.

Model ensemble across architectures adds +0.1–0.3 dB. The NTIRE 2024 winner trained 4 models with different loss functions (L1 and L2) and batch sizes, then applied weighted averaging. [TheCVF](#) ↗ [TheCVF](#) ↗ If training budget permits, train a SwinIR model and a NAFNet model, then average their outputs. Assign weights based on validation performance.

Pseudo-labeling leverages test data. After training your best model, predict HR outputs for all test LR images. Use these (test_LR, predicted_HR) pairs as additional training data. Fine-tune for 10K more iterations at very low learning rate (1×10^{-6}). This is a mild but generally positive technique — the risk of reinforcing errors is low with a well-trained base model.

SRTTA-style test-time adaptation is more aggressive: take each test LR image, degrade it further to create (degraded_LR, test_LR) pairs, and briefly fine-tune the model on these pairs before predicting. [Liner](#) ↗ This achieved **+1.55 dB** over baseline in the original paper, though at significant per-image cost (5.4 seconds per image). [Liner](#) ↗

A practical 12-hour training plan

Given the research, here is the highest-expected-value allocation of your 12-hour GPU budget:

Hours 0–1: Data pipeline setup. Implement online random degradation (blur $\sigma \in [0.5, 3.0]$, noise $\sigma \in [5, 25]$, random downsampling method) applied to HR images each epoch. Add CutBlur with probability 0.5. Implement geometric augmentations (flips + rotations). Optionally convert DIV2K to grayscale as supplemental pretraining data.

Hours 1–2: Pretrained model loading. Load SwinIR grayscale denoising checkpoint. Modify architecture for 2 $\times$ upscaling (change `upscale=1` to `upscale=2`, add PixelShuffle upsampler — load shared body weights with `strict=False`). Set optimizer to AdamW, LR= 2×10^{-5} , cosine annealing to 1×10^{-7} .

Hours 2–8: Main training. Train on 48 $\times$ 48 LR patches for 60K iterations with Charbonnier loss. At iteration 5K, add SSIM loss component (weight 0.15). At iteration 20K, add FFL (weight 0.25). At 60K iterations, switch to 64 $\times$ 64 patches and continue for 40K more iterations. Save checkpoints every 10K iterations.

Hours 8–10: Fine-tuning. Reduce LR to 2×10^{-6} . Train on full 128 $\times$ 128 inputs (no cropping) for 20K iterations. Apply pseudo-labeling: predict test images, add as supplemental training pairs, train 10K more iterations.

Hours 10–11: Ensemble preparation. Average weights of best 3–5 checkpoints (feature ensemble). If time permits, quickly train a second model variant (different loss weights or architecture) for model-level ensemble.

Hours 11–12: Inference. Apply self-ensemble (4–8 geometric augmentations depending on time penalty). Submit.

The combined expected improvement from pretrained fine-tuning (+1–3 dB), CutBlur + augmentation (+0.2–0.5 dB), loss optimization (+0.3–0.5 dB), and ensembling (+0.2–0.4 dB) suggests a realistic total gain of **+2–4 dB PSNR** over your current from-scratch baseline — which, depending on the scoring function's weighting, should translate to a substantial jump from 0.6547 toward the 0.78 range.

Conclusion

The single most impactful change is switching from training from scratch to **fine-tuning a pretrained SwinIR** (or HAT) checkpoint — this alone likely accounts for half the gap to the top score. The second most impactful change is **CutBlur augmentation combined with online degradation diversity**, which is particularly effective when data is scarce. [arXiv](#) [↗] Third, replacing a naive L1 loss with **Charbonnier + SSIM + Focal Frequency Loss** directly optimizes the competition metrics while addressing neural networks' spectral bias. Finally, **self-ensemble TTA and checkpoint weight averaging** provide reliable incremental gains at inference time. [TheCVF](#) [↗] The research consistently shows that these four pillars — pretrained initialization, smart augmentation, composite losses, and ensembling — separate competition winners from the field, and their cumulative advantage is largest precisely when training data is most limited.